

Объектно-реляционное расширение языка SQL

Постреляционные базы данных
Лекция №2

Виноградова М.В.
МГТУ им. Н.Э. Баумана
2026 г.

SQL – Structured Query Language

- 1986 – ANSI – SQL/86 (ISO в 1987)
- 1989 - SQL/86 – запросы и схемы
- 1992 - SQL/92 – схемы, транзакции, соединения, авторизация
- 1995 - SQL/CLI – динамический, ODBC
- 1996 - SQL/PSM – хранимые процедуры
- 1999 – SQL:1999 (SQL3) – объектное расширение, UDT
- 2003 – SQL:2003 – OLAP, XML
- 2006 – XQuery

Основные разделы SQL

- Создание/изменение схемы БД - DDL
- Ограничения целостности и триггеры
- Представления БД
- Структуры физического уровня
- Управление доступом
- Транзакции
- Запросы к данным (CRUD) - DML

Расширение языка SQL для ОРБД

- Поддержка сложных типов данных
- Встроенные (системные) функции
- Хранимые процедуры и функции
- Общие табличные выражения
- Рекурсия
- Ранжирование
- Транспонирование
- Триггеры DDL/DML
- Секционирование

Хранимые процедуры и функции

- PERSISTENT STORED MODULES
- SQL/PSM, PSM-96
- Объекты схемы БД
- CREATE – ALTER – DROP
- Хранятся (в откомпилированном виде) и исполняются на сервере

PSM – объявления

CREATE PROCEDURE имя(параметры)

Объявление локальных переменных

Тело;

CREATE FUNCTION имя(параметры) **RETURNS** тип

Объявление локальных переменных

Тело;

Параметры: режим – имя – тип

- IN (по ум.) | OUT | INOUT
- OUT | INOUT – запрещено для функции

Пример процедуры

```
CREATE PROCEDURE Move  
(  
    IN oldAddress VARCHAR(255),  
    IN newAddress VARCHAR(255)  
)  
UPDATE MovieStar  
SET address = newAddress  
WHERE address = oldAddress;
```

Вызов процедур и функций

- Из хранимой процедуры
- Из программы на базовом языке
- Как команда базового интерфейса SQL

CALL имя(аргументы);

EXECUTE имя(аргументы);

- Функции выделяются как часть крупного выражения (из запроса)

Select * from tab where col in fun(col2)

Типы пользовательских функций

- **Скалярные** – возвращает одно значение
- **Табличные** – возвращают набор записей
- **Встроенные** (inline) – как представление с параметром
- **Агрегатные** – применяют к набору значений. Функции инициализации, итерации, и итогового вычисления.

Возможности PSM

- Объявление и использование переменных
- Возврат значений
- Условия
- Циклы
- Работа с курсором
- Перехват, обработка и вызов исключений
- Вызов системных функций
- DML и DDL команды языка SQL
- Динамические запросы

Операторы PSM

- Возврат значения
RETURN выражение;
- Локальная переменная
DECLARE имя тип;
- Присваивание
SET переменная = выражение;
(= NULL или (SELECT...), если возвращает 1 значение)
- Блоки кода
BEGIN
...
END
- Метки
ИМЯ:

УСЛОВИЯ

IF условие **THEN**

 Список выражений

ELSEIF условие **THEN**

 Список выражений

ELSE

 Список выражений

END IF;

CASE переменная

WHEN значение1 **THEN**

WHEN значение2 **THEN**

END;

Циклы

метка: **LOOP**

код;

условие **LEAVE** метка;

код;

END LOOP;

FOR имя цикла **AS** имя курсора **CURSOR FOR**

Запрос

DO

Список выражений

END FOR;

Циклы - 2

WHILE условие **DO**

Список выражений

END WHILE;

REPEAT

Список выражений

UNTIL условие

END REPEAT;

Курсоры

DECLARE имя **CURSOR FOR** select ...

...

OPEN имя

...

В цикле:

FETCH имя **INTO** переменные;

...

CLOSE имя

```
DECLARE MovieCursor CURSOR FOR SELECT length
FROM Movie WHERE studioName = s;
DECLARE newLen INTEGER;
DECLARE movieCount INTEGER;
BEGIN
    SET movieCount = 0;
    OPEN MovieCursor;
ml: LOOP FETCH MovieCursor INTO newLen;
    IF Not_Found THEN LEAVE ml
    END IF;
    SET movieCount = movieCount + 1;
    SET mean = mean + newLen;
    SET variance = variance + newLen * newLen;
END LOOP;
SET mean = mean / movieCount;
SET variance = variance / movieCount - mean * mean;
```


Объявление переменных в PostgreSQL

- Перед использованием переменной: **declare имя тип**
- Если строковый тип, то необходимо определить структуру таблицы

```
CREATE FUNCTION merge_fields(t_row table1)
    RETURNS text AS
    $$
    DECLARE t2_row table2%ROWTYPE;
    BEGIN
        SELECT * INTO t2_row FROM table2 WHERE ...
    ;
        RETURN t_row.f1 || t2_row.f3 || t_row.f5;
    END;
$$ LANGUAGE plpgsql;

SELECT merge_fields(t.*) FROM table1 t WHERE ... ;
```

Условия в PostgreSQL

```
IF v_count > 0 THEN
    INSERT INTO users_count (count) VALUES (v_count);
    RETURN 't';
ELSE
    RETURN 'f';
END IF;
```

```
SELECT a,
    CASE a WHEN 1 THEN 'one'
           WHEN 2 THEN 'two'
           ELSE 'other' END
FROM test;
```

Циклы в PostgreSQL

LOOP

-- здесь производятся вычисления

EXIT WHEN count > 0;

CONTINUE WHEN count < 50;

-- здесь производятся вычисления

END LOOP;

FOR i IN 1..10 LOOP

-- здесь производятся вычисления

END LOOP;

FOR mviews IN SELECT ... Into mviews

LOOP

-- Здесь "mviews" содержит одну запись

-- Действия с mviews

END LOOP;

Исключения в PostgreSQL

```
INSERT INTO mytab(firstname, lastname)
VALUES('Tom', 'Jones');
BEGIN
    UPDATE mytab SET firstname = 'Joe'
        WHERE lastname = 'Jones';
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'перехватили ошибку';
    -- действия;
END;
```

Пример скалярной функции

```
CREATE FUNCTION add_xy(x integer, y integer)
  RETURNS integer AS
  $$
      SELECT x + y;
  $$
LANGUAGE SQL;
```

Вызов функции:

```
SELECT add_xy(1, 2);
```

Табличные функции

- Функция типа «inline» содержит только один запрос который подставляется в место обращения к функции.
- Табличная функция типа «Multi-statement» из нескольких команд и запросов, вызывается и выполняется как программа.
- Переменная, указанная как возвращаемый тип, содержит набор записей, которые возвращаются в качестве результата функции. Код функции должен обеспечить заполнение этой переменной, например, командой «insert»
 - RETURN NEXT *выражение*;
 - RETURN QUERY *запрос*;
 - RETURN QUERY EXECUTE *строка-команды*;
- Табличная функция указывается в части «FROM» запроса.

Пример табличной функции

```
CREATE OR REPLACE FUNCTION get_all_foo()  
    RETURNS SETOF foo AS  
$BODY$  
DECLARE r foo%rowtype;  
BEGIN  
FOR r IN SELECT * FROM foo WHERE fooid > 0 LOOP  
    -- здесь возможна обработка данных  
    RETURN NEXT r; -- возвращается текущая строка  
запроса  
END LOOP;  
RETURN;  
END;  
$BODY$ LANGUAGE plpgsql;  
  
SELECT * FROM get_all_foo();
```

Пример табличной функции

```
CREATE FUNCTION get_avaiable_flightid(date)  
                RETURNS SETOF integer AS
```

```
$BODY$
```

```
BEGIN
```

```
    RETURN QUERY SELECT flightid FROM flight
```

```
        WHERE flightdate >= $1 AND flightdate < ($1 + 1);
```

```
    IF NOT FOUND THEN
```

```
        RAISE EXCEPTION 'Нет рейсов на дату: %.', $1;
```

```
    END IF;
```

```
    RETURN;
```

```
END;
```

```
$BODY$
```

```
LANGUAGE plpgsql;
```

-- Возвращает доступные рейсы либо вызывает исключение, если их нет.

```
SELECT * FROM get_avaiable_flightid(CURRENT_DATE);
```


Пример процедуры

```
CREATE PROCEDURE triple(INOUT x int)
LANGUAGE plpgsql
AS $$
    BEGIN
        x := x * 3;
    END;
$;
```

```
DECLARE myvar int := 5;
CALL triple(myvar);
```

Динамический запрос

EXECUTE *command-string*

[INTO *target*]

[USING *expression* [, ...]];

EXECUTE 'SELECT count(*) FROM ' ||

tabname::regclass ||

' WHERE inserted_by = \$1

AND inserted <= \$2'

INTO c

USING checked_user, checked_date;

Пример ранжирования (оконные функции)

```
| select ROW_NUMBER() over (order by src),  
       rank() over (order by src),  
       dense_rank() over (order by src),  
       ntile(3) over (order by src),  
       src  
- from roads
```

1	1	1	1	анапа
2	2	2	1	лондон
3	3	3	1	москва
4	3	3	2	москва
5	5	4	2	париж
6	6	5	3	СПб
7	6	5	3	СПб

Представления (view)

- Объекты БД
- Виртуальные таблицы
- Используются наравне с обычными таблицами
- Не содержат данных
- Хранятся в форме откомпилированного запроса
- При обращении выполняется соответствующий запрос

DDL для представлений

Создание

CREATE VIEW Имя [(поля,...)]
AS SELECT ...

Изменение (при сохранении полей)

ALTER VIEW Имя [(поля,...)]
AS SELECT ...

Удаление

DROP VIEW Имя

Пример представления

create view Worker as

select fio,age

from Person where work is not null;

→ Worker(fio, age)

Select * from Worker where age>25;

Пример представления на основе нескольких таблиц

create view Worker as

select Person.*, Org.*

**from Person join Org on
Person.work=Org.title;**

Select * from Worker join City

on Worker.city = City.name

where Worker.post = 'dir' and City.capital = true;

Использование представлений

- Определение
 - На основе любых запросов на выборку
 - На основе любого количество таблиц или представлений из разных БД
- Использование
 - Наравне в таблицами в любой части select – запросов (from, exists, ...)
 - Вместо подзапросов в сложных запросах
 - В любых операциях при соединении таблиц/представлений
 - Назначать на представление индексы и триггеры

Обновляемое представление

- Допускает выполнение команд insert, update, delete
- Основано на одной таблице или обновляемом представлении
- Содержит все обязательные и ключевые поля
- Запрос не содержит операторы по преобразованию строк, например, группировки, агрегирования и т.д.

Триггеры DML

CREATE TRIGGER Название
AFTER UPDATE OF Таблица **ON** Поле
REFERENCING

 OLD ROW AS oldTuple,

 NEW ROW AS newTuple

FOR EACH ROW

WHEN (условие)

BEGIN

 код

END

Парметры DML триггера

AFTER | BEFORE | INSTEAD OF

UPDATE | INSERT | DELETE

WHEN – не обязательно

FOR EACH ROW

-- код работает для кортежа

FOR EACH STATEMENT

-- для всего отношения

OLD TABLE AS имя1

NEW TABLE AS имя2

Пример DML триггера в PostgreSQL

```
CREATE TRIGGER check_update
    BEFORE UPDATE ON accounts
    FOR EACH ROW
    WHEN (OLD.balance IS DISTINCT FROM
NEW.balance)
    EXECUTE FUNCTION check_account_update();
```

```
CREATE TRIGGER transfer_insert
    AFTER INSERT ON transfer
    REFERENCING NEW TABLE AS inserted
    FOR EACH STATEMENT
    EXECUTE FUNCTION check_transfer_balances_to_zero();
```

DDL триггеры

- Уровня БД / уровня сервера
- Изменения структуры объектов БД
- Системные переменные (структуры) для получения информации о событии
- Возможность отката транзакции

Пример DDL триггера в PostgreSQL

```
CREATE FUNCTION snitch()  
  RETURNS event_trigger AS  
  $$ BEGIN  
      RAISE NOTICE 'snitch: % %', tg_event,  
      tg_tag;  
  END; $$  
LANGUAGE plpgsql;
```

```
CREATE EVENT TRIGGER snitch  
  ON ddl_command_start  
  EXECUTE FUNCTION snitch();
```

Общие табличные выражения (пример)

```
WITH regional_sales AS  
      ( SELECT .....),  
top_regions AS  
      ( SELECT .... )  
  
SELECT ...  
FROM regional_sales, top_regions  
WHERE .....
```

Задачи на построение рекурсивных запросов

- Обход дерева или иерархии,
- Нахождение путей и их параметров
- Проверка достижимости

cfrom character v	cto character v	rtype character v	rcost integer	rtime integer
MOS	SPb	RGD	4000	4
MOS	ANAPA	avia	3000	3
ANAPA	Stambul	avia	5000	1
SPb	Riga	avia	1000	1
Riga	London	avia	7000	3
London	NewYork	sea	15000	30
London	Paris	avia	2000	2

Таблица дорог

Roads(cfrom, cto,
rtype, rcost, rlen)

Рекурсивные запросы

```
WITH Recursive Path as          -- вычисляемая таб.  
(  
  select cfrom,cto              -- база вычислений  
  from Roads  
Union  
  select Path.cfrom, Roads.cto  -- индукция  
  from Path, Roads  
  where Path.cto = Roads.cfrom  
)  
select * from Path            -- итоговый запрос
```

Шаг 1 – база вычислений

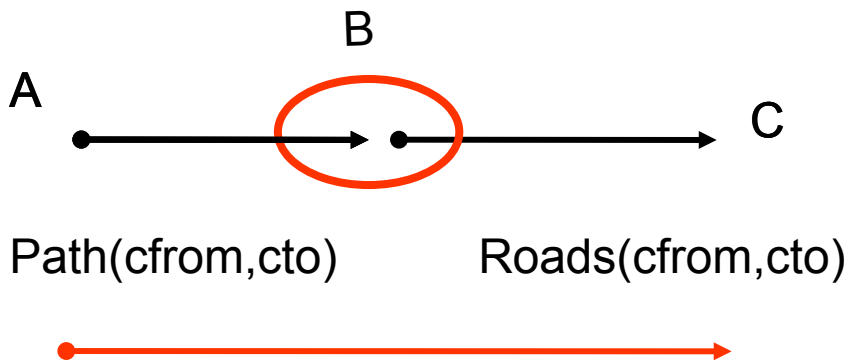
Path(cfrom, cto) заполняем как
`select cfrom, cto from Roads`

cfrom character varying	cto character varying
MOS	SPb
MOS	ANAPA
ANAPA	Stambul
SPb	Riga
Riga	London
London	NewYork
London	Paris

Шаг 2 – индукция - 1

Path(cfrom, cto) заполняем как

`select Path.cfrom, Roads.cto from Path, Roads
where Path.cto = Roads.cfrom`



cfrom character varying(25)	cto character varying(25)
MOS	SPb
MOS	ANAPA
ANAPA	Stambul
SPb	Riga
Riga	London
London	NewYork
London	Paris
MOS	Riga
MOS	Stambul
SPb	London
Riga	Paris
Riga	NewYork

Шаг 3 – индукция - 2

select Path.cfrom, Roads.cto from Path, Roads
where Path.cto = Roads.cfrom

cfrom character varying(25)	cto character varying(25)
MOS	SPb
MOS	ANAPA
ANAPA	Stambul
SPb	Riga
Riga	London
London	NewYork
London	Paris
MOS	Riga
MOS	Stambul
SPb	London
Riga	Paris
Riga	NewYork
MOS	London
SPb	Paris
SPb	NewYork

Шаг 4 – индукция – 3 (останов)

Итоговое
значение
вычисляемого
отношения
Path

cfrom character varying(25)	cto character varying(25)
MOS	SPb
MOS	ANAPA
ANAPA	Stambul
SPb	Riga
Riga	London
London	NewYork
London	Paris
MOS	Riga
MOS	Stambul
SPb	London
Riga	Paris
Riga	NewYork
MOS	London
SPb	Paris
SPb	NewYork
MOS	Paris
MOS	NewYork

Шаг 5 – итоговый запрос

`select * from Path`

<code>cfrom</code> <code>character varying(25)</code>	<code>cto</code> <code>character varying(25)</code>
MOS	SPb
MOS	ANAPA
ANAPA	Stambul
SPb	Riga
Riga	London
London	NewYork
London	Paris
MOS	Riga
MOS	Stambul
SPb	London
Riga	Paris
Riga	NewYork
MOS	London
SPb	Paris
SPb	NewYork
MOS	Paris
MOS	NewYork

Пример рекурсивного запроса

```
WITH Recursive
  Path(cfrom,cto,rcost,rnum) as
(
  select cfrom,cto,rcost,
    1 as rnum
    from Roads
union
  select Path.cfrom,
    Roads.cto,
    Path.rcost + Roads.rcost,
    Path.rnum + 1
    from Path, Roads
   where Path.cto = Roads.cfrom
)
select * from Path
where rcost<20000
```

cfrom character	cto character var	rcost integer	rnum integer
MOS	SPb	4000	1
MOS	ANAPA	3000	1
ANAPA	Stambul	5000	1
SPb	Riga	1000	1
Riga	London	7000	1
London	NewYork	15000	1
London	Paris	2000	1
MOS	Riga	5000	2
MOS	Stambul	8000	2
SPb	London	8000	2
Riga	Paris	9000	2
MOS	London	12000	3
SPb	Paris	10000	3
MOS	Paris	14000	4

Пример рекурсивного запроса

WITH Recursive Path as

(
 select cfrom,cto from Roads

where cfrom = 'MOS' and rtype<>'avia'

union

select Path.cfrom, Roads.cto from Path, Roads
 where Path.cto = Roads.cfrom and rtype<>'avia'

)

select * from Path

cfrom	cto
character	character var
MOS	SPb

Свойства рекурсии

- Обход в ширину или в глубину
- В стандарте – линейная рекурсия (одна ссылка на себя)
- Левосторонняя и правосторонняя (лин.)
- Взаимная рекурсия
- Свойство монотонности (набор не уменьшается при итерации)
- Запрещены в итерации: **distinct, not exists, intersect, except**
- Фиксированная точка – конец итераций
- Избежание циклов

Спасибо за внимание!

Виноградова Мария Валерьевна

Vinogradova.m@bmstu.ru

МГТУ им. Н.Э. Баумана
кафедра Систем обработки информации и
управления (ИУ5)